Assignment

- **Objective** : The Objective of this Assignment to compare the Autoregressive model and the Language Diffusion model from scratch.

Dataset & preprocessing

- From the website Gutenberg.org there contain free eBooks that I downloaded 8 Books and I want to Apply the preprocessing
- In preprocessing table of contents, extra spaces, unnecessary characters all of them are Removed
- Only the text content that I kept in the document that is .txt file
- Now using the tiktoken package here I use the 'Gpt2' Vocabulary around 50257 tokens are there in Gpt2 the Algorithm that Converts the text to tokens Here we use is the Byte pair encoding

- **Byte pair encoding**
  - Start with characters
  - DO Repeatedly
    - → Find the most frequent Adjacent pair
    - → Merge that pair into one token
    - → Continue until Vocabulary Size is Reached
  - Let's do with one example Assume our dataset Contain these words Low, Lowest, Newer, wider

- We split every word into characters and add special End Symbol </w>
- l o w </w>
  l o w e s t </w>
  n e w e r </w>
  w i d e r </w>
- Now count Adjacent pairs

	frequency
(l, o)	2
(o, w)	2
(e, r)	2
(r, </w>)	1
(e, s)	1
(s, t)	1
(t, </w>)	1
(n, e)	1
(e, w)	1
(w, e)	1
(i, d)	1
(d, e)	1

- most frequent pair Lets take (o,w) so merge the most frequent pair

Now Vocabulary becomes

l ow </w>
l ow e s t </w>
n e w er </w>
w i d e r </w>

- Recompute Adjacent pairs

  (l, ow)

  (ow, </w>

  :

- Again find the most frequent pair and merge them

Advantages

- Handles unknowns words
- Small vocabulary
- Better generalization
- efficient for transformers.

- Like this tiktoken done the Byte pair encoding and Create the vocabulary of 50257

GPT Model

- Here we are using the Decoder only transformer
- this means Causal Attention is Applied that is Current token doesnot have the Access of future tokens

Parameters to train GpT model

Vocab_size = 50257

token_dim = 512

num_heads = 8

num_layers = 6

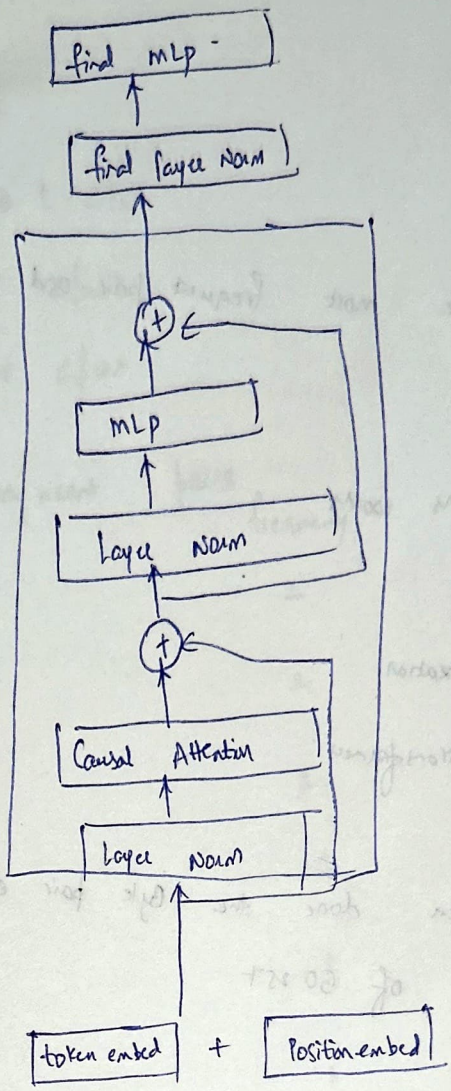
Context_length = 256.

learning_rate = 3e-4

batch_size = 128

epochs = 10

Architecture



- from the Data loader I get [B, context_length] ie [B, 256]
- Now token embedding which is the embedding layer of weights is [vocab_size, token_dim] => [50257, 512]
- We divide each word into tokens so for that token weights from the token embedding
- Coming to the position embedding so the context length is 256 so the position embedding layer weights is [context_length, token_dim] => [256, 512]
- so [B, 256] —token embedding→ [B, 256, 512] —Position embedding→ [B, 256, 512]

Layer Norm :-

- Each Row that contain '512' numbers so mean Applied and Variance Applied
- mean $\mu = \frac{1}{512}\sum_i x_i$
- Variance $\sigma^2 = \frac{1}{512}\sum_i (x_i - \mu)^2$
- $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ 'ϵ' = Very Small value to avoid the division by zero error we do this
- Here these are $\gamma \hat{x}_i + \beta$ [γ and β are the learnable parameters we bought the distribution to the standard Normal but by using 'γ' and 'β' we are telling the model that you can go to some other distribution]
- input to layer Norm $[B, 256, 512]$ $\xrightarrow[\text{layer Norm}]{\text{output After}}$ $[B, 256, 512]$

Causal Attention

- Current token does not have the Access to the Future token
- we Already given that num_heads = 8
- So each head dim => $512/8 = 2^9/2^3 = 2^6 = 64$ is each head dim
- we have the W_q, W_K, W_V matrices $[512,512]$, $[512,512]$, $[512,512]$

$$Q = X \cdot W_q = [B, 256, 512] \cdot [512, 512] = [B, 256, 512]$$

$$K = X \cdot W_K = [B, 256, 512] \cdot [512, 512] = [B, 256, 512]$$

$$V = X \cdot W_V = [B, 256, 512] \cdot [512, 512] = [B, 256, 512]$$

- $[B, num\_tokens, token\_dim]$ ie $[B, 256, 512]$ 2'm changing its shape

$[B, num\_tokens, token\_dim] \longrightarrow [B, num\_tokens, num\_heads * head\_dim]$

ie $[B, 256, 64] = [B, num\_heads, num\_tokens, head\_dim]$

- $\text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$

- Q = [B, num_heads, num_tokens, head_dim]

  K = [B, num_heads, num_tokens, head_dim]

  V = [B, num_heads, num_tokens, head_dim]

Q.K^T

[B, num_heads, num_tokens, head_dim] . [B, num_heads, head_dim, num_tokens]

= [B, num_heads, num_tokens, num_tokens]

ie [B, 8, 256, 64] . [B, 8, 64, 256] = [B, 8, 256, 256]

$\sqrt{d}$ is $\sqrt{64} = 8$ [TO make the Variance close to 1]

Other wise Softmax will Assign Higher probability for higher values

- [B, num_heads, num_tokens, num_tokens] . [B, num_heads, num_tokens, head_dim]

  = [B, num_heads, num_tokens, head_dim] [we multiplied with V]

- then Again reshape to [B, num_tokens, token_dim]

  = [B, 256, 512]

- Now final projection the out matrix is weight matrix

  [512, 512]

  [B, 256, 512] . [512, 512] = [B, 256, 512]

- one thing Before Applying the Softmax future tokens are made to $-\infty$ so that Softmax of $-\infty$ is $\boxed{0}$

- **MLP**
  - The input to the MLP is $[B, 256, 512]$
  - Here the weighted matrix $[512, 2048]$ we are projecting to 4 times the token dimension
  - The Activation function used here is GELU $x \cdot \phi(x)$ where $\phi(x)$ is the Cummulative Distribution function of the Standard Normal distribution
  - then Another weighted matrix $[2048, 512]$ we are projecting Back to token dimension

    $[B, 256, 2048] \cdot [2048, 512] = [B, 256, 512]$

Output layer

- the output of the transformer Block pass through the final layer norm, this layer norm is same as the layer Norm used in the transformer Architecture
- final output layer weights are $[512, 50257]$

  $[token\_dim, vocab\_size]$

  $[B, 256, 512] \cdot [512, 50257] = [B, 256, 50257]$

Loss function

$$L = -\sum_i y_i \log \hat{y}_i$$

multiple tokens are there so Categorical cross entropy

Sampling

- While Sampling we pass the small sentence to the model so model will predict the next token that next token is Added to the Sentence until the Context length Reached

- We are using last token probability and the token with the highest probability get Appended.

- Our model can become deterministic in some cases, model to become probablistic each and every time to generate Different kind of Samples we use Concepts like

  → Temperature

  → top K Sampling

  → multinomial distribution

- **Temperature** :- Once I get the probabilities I divide that probabilities by the Temperature. This temperature can be < 1 or > 1. If the temperature is less than '1' then Sharp distribution, Highest token dominates. If the Temperature greater than '1' more Randomness

  Assume I have '2' tokens token A prob = 0.95 token B = 0.05

  So If the temperature $\neq 1$ Assume $T = 0.5$

  Token A 0.99+ prob

  Token B 0.003 prob

  So token A become more Confident

If the Temperature * 1 ie '2'

Token	Prob
A	0.80
B	0.2

So the prob of B get increased

- **top k Sampling**

multinomial Sampling is Sometimes less probability garabage token there is a chance of selecting to avoid that top 50 Samples are selected these top 50 are High probability tokens So multinomial Can select from this top 50 to avoid selecting the Garbage tokens and SO First temperature Applied then top k Sampling and then multinomial.

- After selecting the top 'k' tokens then Rest all tokens value set to -∞
- Such that ~~probabil~~ Softmax applied to get probability of 0's for the tokens other than top 'k'
- In my code T = 1.0
  K = 50.
- from the Output layer while Sampling we take the last token Output values then Temperature Applied then get the top 50 High Output values Rest all made to -∞ then Softmax Applied to get the probability for that top 50 then multinomial to select the any one probability.